COMP 1405Z Fall 2026
Introduction to Computer Science I
Midterm #2 Oct. 15th, 2026

Midterm Details

You must connect to CoMaS before accessing the midterm resources and remain connected for the duration of the exam. If you suspect there are issues with your CoMaS connection, talk to a proctor or the instructor.

The midterm will be submitted through Brightspace. Put your solutions to each of the problems specified in this document in the corresponding .py file from the provided COMP1405Z-F25-Midterm2-Materials.zip file. To submit your midterm solutions, create a single .zip file with all your midterm materials and submit it to the 'COMP 1405Z Midterm #2' assignment submission on Brightspace. The official end time for the midterm is 11am (Eastern Time) but you will have until 11:10am (Eastern Time) to submit your midterm. **Do not wait until the last minute – no late submissions will be accepted**. After submitting, you should download your own zip file, extract its contents, and ensure the files have the correct content. You are not allowed to consult other students, TAs, the internet, or anybody else while completing the midterm. **You may not consult any existing code examples you have on your computer. The only materials you should access are those included in the midterm materials .zip file. You may use any IDE or text editor you generally use to write your code but must disable any intelligent auto-complete features**. You can execute your programs as many times as you want to debug and fix your code. A test file for each problem is included to help you with testing your implementations. The tests provided provide minimal testing and should not be treated as exhaustive. A Python reference sheet outlining basic operations (string/list/dictionary methods, etc.) has been included in the midterm zip for your use.

Problem 1 (10 marks)

Write the code for this part in the p1.py file. Write a function called sort_cities_by_rainfall(observations). The function accepts one two-dimensional list as its argument. Each entry is a two-element list containing a city name (a string) followed by a rainfall measurement (a number).

The input will contain at least one observation. The same city may appear multiple times. You may assume that all entries follow the specified format. City names are case-sensitive: "Ottawa" and "ottawa" should be treated as different names. Your function must calculate the TOTAL rainfall recorded for each unique city and return a new list containing each city name exactly once. The returned list must be ordered from the city with the LOWEST total rainfall to the city with the HIGHEST total rainfall. Return city names, not observations or rainfall totals. If two cities have the same total rainfall, either ordering of those tied cities is acceptable.

The input list and its inner lists must not be modified. **You may not use Python's built-in sorting functionality, including sorted() and list.sort().**

Example 1:

```
observations = [["Ottawa", 10.0], ["Toronto", 5.0], ["Ottawa", 4.0], ["Kingston", 12.0],
                ["Toronto", 6.0]]
```

Totals: Ottawa = 14.0, Toronto = 11.0, Kingston = 12.0
Return value: ["Toronto", "Kingston", "Ottawa"]

Example 2:

```
observations = [["A", 1.5], ["B", 5.0], ["A", 2.5], ["C", 3.0]]
```

Totals: A = 4.0, B = 5.0, C = 3.0
Return value: ["C", "A", "B"]

Example 3:

```
observations = [["Sudbury", 7.0], ["Ottawa", 2.0], ["Kingston", 5.0]]
```

Totals: Sudbury 7.0, Ottawa = 2.0, Kingston = 5.0
Return value: ["Ottawa", "Kingston", "Sudbury"]

Example 4 (a tie):

```
observations = [["A", 2], ["B", 5], ["A", 3], ["C", 1]]
```

Totals: A = 5.0, B = 5.0, C = 1.0
Return value: ["C", "A", "B"] or ["C", "B", "A"]

Example 5:

```
observations = [["Ottawa", 0.0]]
```

Totals: Ottawa = 0.0
Return value: ["Ottawa"]

Problem 2 (10 marks)

Write the code for this part in the p2.py file. Write a function called find_single_repeat_words(S), which accepts a single string argument S representing some text. The function must return a list containing each word for which EXACTLY ONE DISTINCT CHARACTER occurs more than once. Every other distinct character in that word must occur exactly once. The returned list must not contain any duplicate words.

For example:
- "apple" qualifies because "p" occurs twice, while "a", "l", and "e" each occur once.
- "letter" does not qualify because both "e" and "t" occur more than once.
- "dog" does not qualify because no character occurs more than once.
- "aaaa" qualifies because "a" is the only distinct character in the word and it occurs more than once.

Additional constraints and assumptions:
1. Words are separated by spaces.
2. Letter case must be ignored when determining whether a word qualifies.
3. Duplicate words must be removed without regard to case.
4. All words in the returned list must be lowercase.
5. Words must appear in the returned list in the order in which they first appear in the input text.
6. Punctuation, digits, and other non-letter characters are treated exactly the same as letters. Do not remove punctuation before examining a word.

Example 1: find_single_repeat_words("apple dog letter book") returns ["apple", "book"]

Example 2: find_single_repeat_words("Apple APPLE moon noon committee") returns ["apple", "moon"]

Example 3: find_single_repeat_words("11 a!!a ?? abca abc") returns ["11", "??", "abca"]

Problem 3 (10 marks)

Write the code for this part in the p3.py file. Write a function called same_character_set(word1, word2) that accepts two string arguments and returns True if the two strings contain exactly the same DISTINCT characters, regardless of how many times each character occurs or the order in which the characters occur. Otherwise, the function must return False. You can assume that both input strings contain no spaces and contain only lowercase characters. The number of times a character appears does NOT matter.

Example 1: same_character_set("banana", "ban") returns True
Both strings contain exactly the distinct characters "b", "a", and "n".

Example 2: same_character_set("aabbcc", "abc") returns True
Both strings are made up of only “a”, “b”, and “c”.

Example 3: same_character_set("letter", "tree") returns False
The string "letter" contains "l", but "tree" does not.

Example 4: same_character_set("mississippi", "simp") returns True
Both strings are composed of “s”, “m”, “i”, and “p”

Problem 4 (10 marks)

Write the code for this part in the p4.py file. Write a function called get_ranked_apartments(filename) that accepts a single string argument representing the name of a file. You can assume that the file with the given name contains apartment data in the following repeating six-line pattern:

```
Apartment1_address
Apartment1_neighbourhood
Apartment1_listing_id
Apartment1_monthly_rent
Apartment1_commute_time
Apartment1_size
Apartment2_address
Apartment2_neighbourhood
Apartment2_listing_id
Apartment2_monthly_rent
Apartment2_commute_time
```

```
Apartment2_size
...etc...
```

The listing ID is an integer and every listing ID in the file is unique. Monthly rent, commute time, and apartment size are numbers that may contain decimal values. `monthly_rent` is measured in dollars, `commute_time` is measured in minutes, and `size` is measured in square feet.

For each apartment, calculate a desirability score using the equation below:

desirability = (size * 0.20) – (monthly_rent * 0.03) – (commute_time * 1.00)

A HIGHER desirability score represents a more desirable apartment. Your function must return a list of listing IDs ordered from the apartment with the HIGHEST desirability score to the apartment with the LOWEST desirability score. It is possible that this score calculation may produce a negative number in some cases. Your code must handle this correctly. You do not need to worry about floating point rounding errors: if two apartments have desirability scores within 0.01 of each other, either ordering of those two apartments is acceptable. **You may not use Python's built-in sorting functionality, including sorted() and list.sort().**

Example 1: Suppose apartments_example1.txt contains:

10 Main Street
Centretown
101
2100
25
750
44 Bank Street
Glebe
202
1800
40
900
7 Elgin Street
Downtown
303
2400
15
700

The desirability scores are:

101: 62.0
202: 86.0
303: 53.0

The function should return: [202, 101, 303]

Example 2: If a file contains a single apartment with listing ID 5001, the function should return [5001]

Example 3: If three apartment records have desirability scores:

7001: 92.5
7002: 71.0
7003: 84.25

the function should return: [7001, 7003, 7002]

Submission

Create a zip containing all your midterm resources. Submit your zip to the Brightspace submission for the midterm before the cutoff time of 11:10am. No late submissions will be accepted.

Ensure you make your submission to Brightspace and check-out with a proctor or the instructor before exiting CoMaS.